

Reliability Engineering Project

Cutting Detection Time by 40% and Cascading-Failure Amplification by 35%

A plain-English explanation of every component, the end-to-end workflow, and the architecture

The original one-line summary this document explains

“Reduced mean time to detect 40% and cascading-failure amplification 35% while holding SLOs (99.95% availability, p95 under 250 ms) by instituting error budgets, retries with backoff and jitter, circuit breakers, backpressure, graceful degradation, distributed tracing, and burn-rate dashboards, validated through load testing and fault injection.”

Every idea in that sentence is unpacked below with no assumed background knowledge. Where a number or setting appears, it is a typical, realistic value for a system of this shape; substitute your own where they differ.

Contents

1. The whole project in plain English
2. The words first — a simple glossary
3. The two headline numbers, explained
4. The components, one by one
 - 4.1 Service Level Objectives (SLOs) — the promise
 - 4.2 Error budgets — the allowance
 - 4.3 Timeouts and deadlines — the foundation
 - 4.4 Retries with backoff and jitter
 - 4.5 Circuit breakers
 - 4.6 Bulkheads — isolation of resources
 - 4.7 Backpressure and load shedding
 - 4.8 Graceful degradation
 - 4.9 Distributed tracing
 - 4.10 Burn-rate dashboards and alerts
 - 4.11 Load testing
 - 4.12 Fault injection
5. The workflows, end to end
 - 5.1 Workflow A — how the project itself was built
 - 5.2 Workflow B — the life of a single request
 - 5.3 Workflow C — an incident, minute by minute
 - 5.4 Workflow D — the ongoing weekly loop
6. Architecture diagrams and a walk through every element
7. How the improvements were actually measured
8. Reference: the settings used, and why
9. Mistakes this design deliberately avoids
10. How to explain this project out loud

1. The whole project in plain English

Imagine a large online shop built from many small programs (called services) that talk to each other constantly. One service takes orders, another handles payment, another runs search, another suggests products. They all lean on shared things like a database and a cache.

Two problems kept hurting this kind of system.

- **Problem 1 — we found out too late.** When something broke, nobody knew for many minutes. The first sign was often a customer complaint or an engineer noticing an odd graph. All that time, users were suffering while the team was still unaware.
- **Problem 2 — one small failure became a big failure.** If one service slowed down, the services calling it piled up waiting for it, so they slowed down too, and the ones calling those slowed down as well. A single sick component dragged the whole shop down. That spreading effect is called a cascading failure.

This project fixed both problems, and did it without letting the service get slower or less available than the promises made to users.

What was done, in one paragraph

First, a clear written promise was set for how reliable and how fast the system must be (the SLO), and that promise was turned into a spendable allowance for failure (the error budget). Second, the system was instrumented so every request leaves a trail (distributed tracing) and so the health of the promise is measured continuously; a dashboard then shows how fast the allowance is being spent (burn-rate), and alerts fire off that measurement instead of off vague machine statistics. That is what made detection fast. Third, a set of protective controls was added around every call between services — timeouts, retries that wait politely and randomly, circuit breakers, isolated resource pools, backpressure, and the ability to serve a reduced-but-usable answer. That is what stopped failures from spreading. Finally, none of it was taken on trust: the system was pushed with realistic load tests and deliberately broken with fault injection to prove each control actually behaves as designed.

The result, in plain terms

What changed	Before	After	What it means for a person
Time to notice a problem	Slow, human-triggered	40% faster	Problems get worked on sooner, so fewer users are affected.
Spread of a failure	One fault took several services down	35% less spread	A broken part stays broken on its own instead of taking down the shop.
Availability	99.95% target	99.95% held	The system did not become less reliable while being changed.
Speed (p95)	Under 250 ms target	Under 250 ms held	Users did not pay for the safety features with a slower site.

The last two rows matter as much as the first two. It is easy to make a system safer by making it slower or by turning things off. Holding the speed and availability promises while improving detection and containment is the actual achievement.

A useful mental picture

Think of a hospital emergency department.

- **The promise (SLO):** 95 out of 100 patients are seen within 15 minutes, and the department is open 99.95% of the time.
- **The allowance (error budget):** the small number of patients who may wait longer before the promise is considered broken.
- **Triage and monitors (tracing and burn-rate dashboards):** staff can see at a glance who is deteriorating and how fast, rather than waiting for someone to shout.
- **Fixed time per patient (timeouts):** no single difficult case is allowed to occupy a doctor forever.
- **Closing a ward that is contaminated (circuit breaker):** stop sending patients to a place that cannot help them right now.
- **The waiting-room limit (backpressure):** when the department is full, new arrivals are told honestly to go elsewhere instead of joining a queue that will never move.
- **Treating the urgent, deferring the routine (graceful degradation):** in a crisis, the non-essential clinic closes so the emergency work continues.
- **Fire drills (load testing and fault injection):** the plan is rehearsed on a quiet day, not discovered during a real emergency.

2. The words first — a simple glossary

Every later section uses these words. Nothing here requires prior knowledge.

Term	Plain meaning
Service	One small program that does one job, for example ‘take an order’. A big system is dozens of these talking to each other.
Request	One question sent to a service, for example ‘show me this product page’. The service sends back a response.
Dependency	Anything a service needs in order to answer: a database, a cache, another service, an outside company’s API.
Latency	How long a request takes to be answered, measured in milliseconds (ms). 1,000 ms = 1 second.
p95 latency	Sort every request by how long it took. The p95 is the time the 95th out of 100 took. It describes the slow-ish experience, not the lucky-fast one.
Availability	The share of requests that were answered successfully. 99.95% means 5 failures allowed per 10,000 requests.
SLI	Service Level Indicator: the actual measurement, for example ‘98.7% of requests succeeded in the last hour’.
SLO	Service Level Objective: the target the SLI must stay above, for example ‘at least 99.95%’. An internal promise.
SLA	Service Level Agreement: a promise written into a contract, usually with money attached if it is broken. Looser than the SLO on purpose.
Error budget	The failure the SLO allows. If the target is 99.95%, the budget is the remaining 0.05%. It can be spent.
Burn rate	How fast the error budget is being spent right now, compared to the pace that would use it up exactly on schedule.
MTTD	Mean Time To Detect: the average gap between a problem starting and someone or something noticing it.
MTTR	Mean Time To Recover: the average gap between the problem starting and the system being healthy again.
Cascading failure	A failure that spreads. One component gets sick, and the components depending on it get sick too, and so on outward.
Amplification	How much a failure grows as it spreads. One broken thing causing five broken things is worse than one causing two.
Retry storm	Everyone re-sending failed requests at once, which adds load to a system that is already struggling and makes it worse.
Thundering herd	Many clients waking up and acting at the exact same instant, creating a spike the system cannot absorb.

Term	Plain meaning
Idempotent	An operation that is safe to do twice. 'Set the address to X' is safe to repeat; 'charge the card' is not, unless it carries a unique key.
Trace / span	A trace is the full story of one request across all services. A span is one chapter: a single hop, with its own start and end time.
Fallback	A cheaper, reduced answer served when the normal answer cannot be produced, for example a slightly stale cached page.
Blast radius	How much of the system an experiment or a failure is allowed to touch.

3. The two headline numbers, explained

3.1 “Mean time to detect, reduced 40%”

Mean time to detect (MTTD) is a stopwatch. It starts the moment the system genuinely begins to misbehave, and it stops the moment a human or an automatic alert becomes aware of it. It does not include fixing; that is a different stopwatch (MTTR).

A 40% reduction means that if problems used to be noticed after roughly ten minutes on average, they are now noticed after about six. Those four minutes are not paperwork — they are minutes in which real users are being failed, and minutes in which a small local problem has time to become a large spreading one. Faster detection therefore improves the second number too.

Worked example

Before: 12 incidents in a quarter. Detection times, in minutes: 4, 6, 7, 8, 9, 10, 11, 12, 14, 15, 18, 24. Mean = 11.5 minutes.

After: 12 incidents. Detection times: 2, 3, 3, 4, 5, 6, 6, 7, 8, 9, 11, 19. Mean = 6.9 minutes.

Improvement = $(11.5 - 6.9) \div 11.5 = 40\%$.

Note that the last incident is still slow in both lists. Averages improve because the common cases move, not because every case becomes perfect.

What actually produced the reduction:

- Alerts now fire on what the user experiences (are requests succeeding, are they fast) rather than on machine statistics such as CPU, which are only loosely related to user pain.
- The burn-rate rule evaluates over a short window as well as a long one, so a severe problem is caught in minutes rather than after a long averaging period.
- Noisy, meaningless alerts were deleted. When a pager is trusted, it is answered quickly; when it cries wolf, people wait to see if it resolves itself.
- Fault injection measured detection time directly and repeatedly, which turned MTTD into something that could be tuned rather than something that was merely reported after the fact.

3.2 “Cascading-failure amplification, reduced 35%”

Amplification answers the question: when one thing breaks, how many other things break because of it?

It needs a definition to be measurable. Two are commonly used, and both were tracked here:

- **Service amplification** = (number of services that breached their own SLO during the incident) ÷ (number of services that were the actual root cause). If one sick dependency caused five services to breach, the amplification is 5.
- **Traffic amplification** = (peak requests hitting the sick dependency during the incident) ÷ (its normal request rate). Retries and impatient callers are what inflate this; three tiers each retrying three times turns one user request into up to twenty-seven internal calls.

Worked example

Before: a single slow dependency caused, on average, 5.0 services to breach their SLO. Peak traffic into it reached 6.5× normal because every layer was retrying.

After: the same class of fault caused 3.25 services to breach, and peak traffic into it reached about 1.3× normal because retries were capped, budgeted and jittered, and because breakers stopped sending traffic at all.

Service amplification improvement = $(5.0 - 3.25) \div 5.0 = 35\%$.

This is measured most reliably during fault-injection runs, where the root cause is known exactly because it was the thing deliberately broken.

3.3 “While holding SLOs”

This clause is the constraint that makes the work honest. Both improvements had to be achieved without breaking the existing promises.

The promise	Written precisely	What it costs if missed
99.95% availability	Over a rolling 30-day window, at most 0.05% of eligible requests may fail. That is 5 in 10,000, or about 21.6 minutes of total outage per 30 days.	Users cannot complete purchases; trust and revenue are lost.
p95 latency under 250 ms	At least 95 out of every 100 requests must be fully answered within 250 milliseconds, measured at the edge where the user actually waits.	The site feels sluggish; abandonment rises even when nothing is technically ‘down’.

Two details are easy to miss and both matter:

- **Rolling window, not calendar month.** A rolling 30-day window means the budget recovers gradually every day rather than resetting to full on the first of the month, which would encourage risky behaviour late in a month.
- **Measured at the edge.** Measuring latency inside a service flatters the numbers. Measuring at the gateway includes queueing, retries and internal hops — that is what the user actually waits for.

4. The components, one by one

Each component below is described in the same shape: what it is, why it exists, exactly how it works step by step, the settings that were used, and what would go wrong without it.

4.1 Service Level Objectives (SLOs) — the promise

What it is. A written, measurable target for how good the service must be, agreed with the people who own the product, not invented by engineers alone.

Why it exists. Without a number, ‘reliable’ is an opinion. Every argument about whether to ship a feature or fix a flaw becomes a debate between loud people. With a number, the same argument becomes arithmetic.

How it works, step by step:

- Pick the critical user journeys — the handful of things that must work, such as browse, add to basket, pay. Do not write an SLO for every endpoint; you will drown.
- For each journey choose an indicator (SLI) that reflects the user’s experience. Two are used here: the share of requests answered successfully, and the share answered within a time limit.
- Define exactly what counts. A request is ‘good’ if the response status is not a server error and the answer arrived within the deadline. Requests rejected because the caller sent nonsense (a 400) are excluded, because that is the caller’s fault, not the service’s.
- Decide where it is measured. Here: at the API gateway, because that is the closest point to the user that we control.
- Choose the target and the window: 99.95% and p95 under 250 ms, over a rolling 30 days.
- Publish it. An SLO nobody has seen does not change behaviour.

Why p95 and not the average. Averages hide pain. If 90 requests take 50 ms and 10 take 3 seconds, the average is a comfortable 345 ms while one user in ten is having a terrible time. Percentiles describe real experiences: p95 is the experience of the unluckiest 1 in 20.

Why not aim for 100%. The cost of each extra nine grows steeply, and beyond a point users cannot tell the difference because their own network is less reliable than your service. Chasing 100% also freezes all change, and most outages come from changes that were needed anyway.

Without it. There is no way to say whether the system is healthy, so there is nothing an alert can meaningfully fire on, and no way to prove that the reliability work did not make things worse.

4.2 Error budgets — the allowance

What it is. The failure the SLO permits, treated as a spendable resource. $100\% - 99.95\% = 0.05\%$. If the service handles 100 million eligible requests in 30 days, the budget is 50,000 failed requests.

Why it exists. It converts reliability from a mood into a balance. Some budget is meant to be spent — that is how you buy the right to release changes, run experiments, and deliberately inject faults.

How it works, step by step:

- Compute the budget for the rolling window: total eligible requests $\times$ 0.05%.

- Every failed or too-slow request subtracts from it. This is calculated continuously from the same measurements that back the SLO.
- The remaining budget is shown as a percentage on a dashboard everyone can see.
- A written policy attaches consequences to the balance — this is the part that gives it teeth.

Budget remaining	What the team is allowed to do	Reasoning
Above 50%	Ship normally. Run risky experiments and fault injection freely.	There is room to absorb mistakes; going slower would waste the allowance.
25% – 50%	Keep shipping, but reliability fixes move to the top of the backlog. Risky experiments need a reason.	The trend is the warning, not the crisis.
Below 25%	Only low-risk changes. Fault injection paused in production. Reliability work becomes the main work.	Protect what is left of the promise.
Exhausted	Feature freeze. Only fixes that restore reliability ship until the budget recovers.	The promise has already been broken; adding risk breaks it further.

The most important property. The policy is agreed before it is needed. Deciding a freeze rule during an outage is negotiation under pressure; deciding it in advance is engineering.

Without it. Reliability work is always the thing that gets pushed to next quarter, and there is no agreed signal for when to stop adding features and start repairing.

4.3 Timeouts and deadlines — the foundation everything else sits on

What it is. A timeout is a hard limit on how long one call may wait before it is given up on. A deadline is the same idea for a whole request: a wall-clock moment by which the answer is worthless, passed along to every service the request touches.

Why it exists. Waiting is not free. A call that is waiting is holding a thread, a connection, and a slice of memory. When a dependency slows down, callers do not fail — they wait, and their resources fill up with waiting. That is the exact mechanism by which slowness spreads and becomes a cascade.

How it works, step by step:

- The gateway stamps every incoming request with a deadline, for example ‘answer by 800 ms from now’.
- The deadline travels with the request in a header, so every downstream service knows how much time is genuinely left, not how much it would like to have.
- Before making any outbound call, a service compares the time remaining with the timeout it would normally use, and takes the smaller of the two.
- If no time is left, the call is not attempted at all. Work whose answer can no longer be used is pure waste, and doing it starves work that is still useful.
- Timeouts are set from measured behaviour: typically just above the dependency’s normal p99, so that a timeout means ‘genuinely abnormal’ rather than ‘slightly unlucky’.

- Timeouts get shorter as you go deeper. If the caller allows 800 ms, an inner call cannot sensibly be given 1,000 ms.

Common mistake. Default library timeouts are often measured in minutes, or absent entirely. An unset timeout is not 'patient' — it is an unbounded promise to wait, and it is the single most common root cause of a cascade.

Without it. Retries, breakers and bulkheads cannot work properly, because none of them can tell the difference between 'slow' and 'never going to answer'.

4.4 Retries with exponential backoff and jitter

What it is. When a call fails in a way that might be temporary, try again — but wait longer between each attempt (backoff), and add a random amount to that wait (jitter).

Why it exists. Many failures are momentary: a brief network blip, a server restarting, a short lock. Retrying rescues those requests invisibly. Retrying carelessly, however, is one of the best ways to turn a small problem into an outage.

The three ideas, separately

- **Retry:** attempt the call again instead of failing the user immediately.
- **Backoff:** wait longer each time — for example 100 ms, then 200 ms, then 400 ms (each wait doubles). This gives the struggling dependency progressively more room to recover rather than hammering it.
- **Jitter:** multiply or offset that wait by a random amount, so that a thousand clients that failed at the same instant do not all come back at the same instant. Without jitter, backoff simply synchronises everyone into neat, destructive waves.

The jitter formula used

`wait = random_between(0, min(cap, base × 2attempt))`

With base = 100 ms and cap = 2,000 ms: attempt 1 waits a random time between 0 and 200 ms; attempt 2 between 0 and 400 ms; attempt 3 between 0 and 800 ms.

This is known as ‘full jitter’. It spreads the returning traffic evenly instead of leaving spikes at 200 ms, 400 ms and 800 ms exactly.

The rules that keep retries from becoming the outage

- **Only retry what is safe to repeat.** Reads are safe. Writes are only safe if they carry an idempotency key so the receiver can recognise and ignore a duplicate. Charging a card twice is worse than failing once.
- **Only retry retryable errors.** A timeout, a connection reset or a 503 may be temporary. A 400 (bad request) or a 403 (not allowed) will fail identically every time; retrying it is pure waste.
- **Cap the attempts.** Two extra attempts at most. Unlimited retries are just a slow denial-of-service attack against your own dependency.
- **Retry at exactly one layer.** This is the big one. If three layers each retry three times, one user request becomes up to twenty-seven calls at the bottom — a 27× amplification created entirely by well-meaning code. Retrying is enabled at one agreed layer and disabled everywhere else.
- **Keep a retry budget.** Across the whole service, retries may not exceed roughly 10% of normal request volume. When that ceiling is hit, retrying switches off globally until traffic settles. This is what prevents a retry storm even if every individual rule was followed.
- **Never retry past the deadline.** If the request’s deadline is 40 ms away, there is no point starting an attempt that needs 200 ms.
- **Do not retry into an open circuit.** If the breaker has already decided the dependency is unwell, retrying is arguing with the diagnosis.

Contribution to the results. This single component is responsible for a large share of the amplification improvement, because uncontrolled retries are the main mechanism by which load multiplies during an incident.

4.5 Circuit breakers

What it is. A component that watches the recent success rate of calls to one dependency and, when the failure rate is bad enough, stops sending calls entirely for a while. Named after the electrical breaker in a house, which cuts the circuit rather than letting a fault burn the wiring.

Why it exists. When a dependency is genuinely down or overloaded, every call to it is doomed — but each doomed call still costs the caller a thread and a connection for the full timeout duration. Thousands of doomed calls exhaust the caller's resources, and the caller then fails for everyone, including for requests that had nothing to do with the sick dependency.

The three states

State	Behaviour	How it moves to the next state
Closed (normal)	Calls pass through as usual. Successes and failures are counted in a rolling window.	If the failure rate crosses the threshold (and enough calls have been seen), it opens.
Open (tripped)	No calls are attempted at all. Every call fails instantly and the fallback runs. This costs almost nothing and sends zero traffic to the sick dependency.	After a cool-off period, typically 30 seconds, it moves to half-open.
Half-open (testing)	A small number of trial calls are allowed through, for example five.	If the trials succeed, it closes and normal service resumes. If any fail, it opens again and the cool-off restarts.

How it works, step by step:

- Each caller keeps one breaker per dependency — not one breaker for everything, or a single sick database would block calls to a healthy cache.
- A rolling window counts outcomes, for example the last 20 calls or the last 10 seconds.
- A minimum sample size is required before the breaker may trip, so two unlucky failures on a quiet morning do not disable a healthy dependency.
- Slow calls count as failures too. A dependency answering correctly but 10× too slowly is just as dangerous as one that is down.
- When it trips, the state change is recorded as an event and appears on the dashboard — which is itself an early detection signal, often the first sign of trouble.
- While open, the fallback path serves users a reduced answer instead of an error, which is how availability is protected during the trip.

The counter-intuitive part. A breaker makes some requests fail on purpose, faster. That sounds like it should hurt availability. It protects it, because the alternative is every caller freezing up and failing for everyone. Failing 5% of requests immediately is far better than failing 100% of them slowly.

Without it. The classic cascade: a slow dependency fills the caller's thread pool, the caller stops answering anyone, its callers fill up in turn, and the failure walks all the way to the front door.

4.6 Bulkheads — isolating resources

What it is. Separate pools of threads and connections for each dependency, so that work waiting on one dependency cannot consume the resources needed by another. Named after the watertight compartments in a ship's hull: one flooded compartment does not sink the vessel.

How it works, step by step:

- Each dependency is given its own fixed allocation, for example 20 concurrent calls to the payment provider and 40 to the database.
- When a pool is full, further calls to that dependency are rejected immediately rather than queued indefinitely — this is backpressure applied at the smallest scale.
- Pools are sized from load-test measurements, not guessed.
- Optional traffic gets smaller pools than critical traffic, so recommendations can never starve checkout.

Why it matters here. Bulkheads are what make the circuit breaker's job survivable in the seconds before it trips. Even during that window, the damage is confined to one pool.

Without it. One shared thread pool means every dependency shares one fate. A slow, unimportant dependency can take down the most important path in the system.

4.7 Backpressure and load shedding

What it is. Backpressure is the system pushing back when it is overloaded: telling callers honestly ‘I cannot take this right now’ instead of accepting work it has no chance of completing. Load shedding is the act of dropping some work on purpose so that the rest can succeed.

Why it exists. A system that accepts everything under overload does not go faster — it goes slower, then it dies. Requests pile up in queues; by the time each one is picked up, the caller has already given up; the server then spends all its capacity producing answers nobody is waiting for. This is the failure mode where a system appears to be working hard while achieving nothing.

Why queues do not save you

Little’s Law: average queue length = arrival rate × average wait. Rearranged: wait = queue length ÷ throughput.

If a service handles 500 requests per second and 1,000 requests are queued in front of it, the last one in that queue waits 2 seconds — long after its 800 ms deadline has expired.

An unbounded queue therefore does not add capacity. It converts a fast, honest failure into a slow, useless one, and it guarantees that every answer is produced too late to matter.

How it works, step by step:

- Every queue is bounded. A fixed maximum depth is set, sized so that the wait at maximum depth is still inside the deadline.
- Concurrency limits cap how many requests a service will work on at once. The limit comes from load testing: the point past which extra concurrency adds latency without adding throughput.
- When the limit or the queue is full, the request is rejected immediately with a clear ‘try again later’ status (HTTP 429 or 503) and a Retry-After hint telling the caller roughly how long to wait.
- Rejection happens at the edge wherever possible. Refusing a request at the gateway costs almost nothing; refusing it after five internal hops has already spent the resources it was meant to save.
- Shedding is prioritised. Optional traffic is dropped first, then background jobs, then bulk API traffic, and paying users completing a purchase are dropped last.
- Queued work whose deadline has already expired is discarded before it is started, rather than being processed pointlessly.
- For asynchronous paths, the queue itself becomes the buffer: writes are accepted onto a durable message queue and applied when the downstream system can cope, so the user is served immediately while the work is completed shortly afterwards.

The honest trade. Load shedding means deliberately failing some requests. It is chosen because the alternative is failing all of them. The SLO makes this defensible: shedding 1% for two minutes may cost a slice of the error budget, whereas a full collapse costs all of it plus hours of recovery.

4.8 Graceful degradation

What it is. The ability to serve a reduced but still useful answer when the perfect answer cannot be produced — instead of serving an error page.

Why it exists. Users do not need everything; they need the important thing. A product page without personalised recommendations is still a product page. An error message is not.

Features ranked in advance

The ranking is the real work, and it is done calmly beforehand, with the product owner, not during an incident.

Tier	Examples	What happens under stress
Critical	Browse, add to basket, checkout, payment	Never degraded. Everything else is sacrificed to protect these.
Important	Search, order history, stock levels	Falls back to cached or approximate data; slightly stale is acceptable.
Optional	Recommendations, 'people also bought', banners, reviews summary	Switched off first, silently. The page renders without the section.

The degradation techniques used:

- **Serve stale from cache.** Return the last known-good answer with a note of its age. A price from 90 seconds ago is nearly always better than no page at all.
- **Return partial results.** Render the page with the sections that succeeded and omit the ones that did not, rather than failing the whole page because one panel failed.
- **Simplify the computation.** Swap a personalised ranking for a generic bestseller list — same shape of answer, a fraction of the cost.
- **Defer the work.** Accept the order onto a queue and confirm it asynchronously, instead of holding the user while a struggling downstream system is waited on.
- **Read-only mode.** When writes are unsafe, keep the site browsable rather than taking it down entirely.

How it is controlled:

- Every optional feature sits behind a feature flag — a switch that can be turned off in seconds without a code deployment.
- Some switches are automatic: when the circuit breaker for recommendations opens, the fallback is used with no human involved.
- Some are manual: the on-call engineer can shed a whole feature tier from a control panel, following the runbook.
- Fallbacks must be cheap and must not touch the sick dependency. A fallback that calls the same broken database is not a fallback.
- Fallback paths are exercised regularly in testing, because an untested fallback is usually a broken fallback — it is the code path nobody runs on a normal day.

4.9 Distributed tracing

What it is. A way of following one single request through every service it touches, with a timestamp at every hop, so that the whole journey can be read afterwards as one connected story.

Why it exists. In a system of thirty services, logs tell you that many things happened, and metrics tell you that something is wrong somewhere. Neither tells you which hop of which journey is slow. Tracing answers that question directly, and answering it quickly is most of what fast detection and fast triage mean in practice.

How it works, step by step:

- When a request arrives at the gateway it is given a unique trace ID — a random identifier for this one journey.
- The trace ID travels in a request header (the W3C ‘traceparent’ standard) to every service the request touches, including through message queues.
- Each service records a span: a small record saying ‘I started this piece of work at time X, finished at time Y, this was my parent, and here is the outcome’.
- Spans are sent to a collector, which reassembles them by trace ID into a waterfall diagram: one horizontal bar per hop, laid out in time.
- Reading that waterfall makes the slow hop obvious. The longest bar is the answer, and its position shows whether it was the cause or a victim.
- Sampling keeps the cost sane: a small percentage of normal traffic is kept, but every failed or unusually slow request is kept, because those are the interesting ones.
- Traces and metrics are linked. From a spike on a latency graph, one click reaches an example trace from that exact moment — which is what turns ‘something is slow’ into ‘the payment provider’s TLS handshake is slow’ in seconds.

What it adds beyond metrics. Metrics say a service is slow. Traces say why, and where. Metrics are the smoke alarm; traces are the floor plan showing which room is burning.

Contribution to the results. Tracing does not by itself make an alert fire sooner, but it collapses the time between the alert and the correct diagnosis — and in the measured pipeline, an incident is only counted as detected when the on-call has confirmed a real problem, so faster confirmation moves MTTD directly.

4.10 Burn-rate dashboards and alerts

What it is. A dashboard and an alerting rule based on how fast the error budget is being consumed, rather than on raw failure counts or machine statistics.

The core idea. Burn rate is a multiplier. A burn rate of 1 means the budget is being spent at exactly the pace that would use it all up over the 30-day window — sustainable, if unpleasant. A burn rate of 10 means it is being spent ten times too fast, and the entire month’s allowance will be gone in three days.

The arithmetic, worked through

Target 99.95% → the budget is 0.05% of requests, over 30 days.

If 0.05% of requests are failing right now, the burn rate is 1.0 — the budget lasts exactly 30 days.

If 0.72% of requests are failing right now, the burn rate is $0.72 \div 0.05 = 14.4$ — the entire 30-day budget would be gone in about 50 hours. That is a genuine emergency, and it is worth waking someone up.

If 0.15% are failing, the burn rate is 3.0 — the budget lasts 10 days. Serious, but it can wait for business hours.

The alerting rules used:

Severity	Rule	Meaning in plain terms	Action
Page (urgent)	Burn rate above 14.4 sustained over 1 hour, confirmed by a 5-minute short window	2% of the monthly budget consumed in a single hour	Wake someone up now
Page (high)	Burn rate above 6 sustained over 6 hours, confirmed by a 30-minute window	5% of the budget gone in six hours	Wake someone up if out of hours
Ticket (medium)	Burn rate above 3 over 24 hours, confirmed by a 2-hour window	10% of the budget gone in a day	Work it during the day
Ticket (low)	Burn rate above 1 over 3 days	The slow leak that would breach the SLO by month end	Put it in the backlog

Why two windows for each rule:

- The long window decides whether the problem is real and sustained, which prevents alerts firing on a one-second blip.
- The short window decides whether it is still happening right now, which means the alert resolves quickly after the problem stops instead of nagging for an hour.
- Both must be true at once. This combination is what makes the alerts trustworthy — and trustworthy alerts are answered immediately, which is a large part of the detection improvement.

What the dashboard shows, top to bottom:

- Budget remaining as a single large percentage, with the trend over the window.
- Current burn rate on each of the windows, colour-coded against the thresholds.
- The two SLIs themselves: success rate and p95 latency, per critical journey.
- A breakdown by service and by endpoint, so that the largest contributor to the burn is visible immediately.
- Circuit-breaker state changes, shed counts and retry counts overlaid on the same timeline, so cause and effect line up visually.

Why this beats ‘CPU above 80%’ alerts. High CPU is often perfectly fine, and plenty of severe outages happen at low CPU. Alerting on machine statistics produces alerts that are frequently wrong in both directions, which trains people to ignore the pager. Alerting on the user-visible promise produces alerts that are almost always worth answering.

4.11 Load testing

What it is. Deliberately sending large volumes of realistic traffic at the system to find out where it stops coping — before real users find out for you.

Why it exists. Every number in the previous sections — pool sizes, concurrency limits, queue depths, timeouts — is either measured or guessed. Guessed limits are usually wrong in the dangerous direction.

How it works, step by step:

- Build a traffic model from real production patterns: the actual mix of endpoints, the actual ratio of reads to writes, realistic payload sizes, and realistic think-time between user actions.
- Reproduce the shape of peak traffic, not a flat line. Real traffic arrives in bursts, and bursts are what break things.
- Ramp the load up gradually and watch for the knee — the point where latency starts climbing steeply while throughput stops rising. That knee is the true capacity.
- Set concurrency limits and pool sizes just below the knee, so the system refuses work before it enters the region where it falls over.
- Run a soak test: hold moderate load for several hours to expose slow leaks, growing queues and connection exhaustion that a ten-minute test would never reveal.
- Run a spike test: jump instantly from low to very high load to confirm that admission control and shedding engage in time.
- Record the results as the baseline, and re-run them for every release so that a capacity regression is caught as an ordinary test failure.

Where it runs. In an environment sized like production, or against production itself using a small percentage of shadow traffic — traffic that is copied and replayed but whose responses are thrown away, so nothing real is affected.

4.12 Fault injection

What it is. Deliberately breaking something in a controlled way to confirm that the protective controls behave as designed. Also called chaos engineering.

Why it exists. Every control described above is a claim: ‘the breaker will trip’, ‘the fallback will serve’, ‘the alert will fire within two minutes’. An untested claim about failure behaviour is usually false, because the failure path is the one path that never runs on a normal day.

How an experiment is run, step by step:

- State a hypothesis in advance and in writing: ‘if the payment provider adds two seconds of latency, checkout will degrade to asynchronous confirmation, the breaker will open within 30 seconds, no other service will breach its SLO, and the alert will fire within two minutes’.
- Choose the blast radius: one service, one region, or a small percentage of traffic. Start in staging, then move to production with the smallest slice that still produces a real signal.
- Announce it and arm a kill switch that aborts the experiment instantly.
- Inject exactly one fault at a time, so the result is attributable. Common injections: added latency, a percentage of errors, a killed process or node, packet loss, a cold cache, a corrupt response, or a whole region withdrawn.
- Measure against the hypothesis: how long until the alert fired (this is a direct MTTD measurement), how many services breached (this is a direct amplification measurement), whether users still got a usable answer, and whether the system healed itself once the fault was removed.
- Fix whatever failed the hypothesis, then re-run the identical experiment to prove the fix.

- Keep every experiment as a repeatable regression test, and run them on a schedule, including as periodic ‘game days’ where the on-call rotation practises the response.

Why this matters for the headline numbers. Fault injection is the only reason the two improvement figures can be stated with confidence. In production incidents you rarely know the counterfactual; in an injection run, the cause is known exactly because it was chosen, so before-and-after comparisons are genuinely like for like.

5. The workflows, end to end

There are four different workflows in this project, and they are often confused with each other. Workflow A is how the project was built. Workflow B is what happens to a single request on a normal day. Workflow C is what happens during an incident. Workflow D is the routine that keeps it all honest over time.

5.1 Workflow A — how the project itself was built, phase by phase

The order matters. Each phase makes the next one possible, and doing them out of order is the usual reason this kind of work fails.

Phase 0 — Establish the baseline (you cannot improve what you have not measured)

- Pull the last two quarters of incidents and, for each, record when it actually started and when it was actually noticed. This produces the ‘before’ MTTD figure.
- For each incident, count how many services breached during it and how many were the root cause. This produces the ‘before’ amplification figure.
- Write down where the time went: how long until anyone noticed, how long to find the cause, how long to fix. Detection and diagnosis usually dominate, which is what justifies investing in tracing and burn-rate alerts first.
- Map the dependencies: which service calls which, and which of those calls are on the critical path for a purchase.

Phase 1 — Agree the promise

- Pick the critical user journeys with the product owner.
- Define the two SLIs precisely, including exactly which responses count as failures and where the measurement is taken.
- Set the targets: 99.95% availability and p95 under 250 ms over a rolling 30 days.
- Write the error-budget policy, including the freeze rule, and get it agreed in writing before it is ever needed.

Phase 2 — See what is happening

- Add tracing to every service, propagate the trace ID and the deadline through every hop, including across message queues.
- Emit consistent metrics everywhere: request counts, success and failure counts, and latency histograms per endpoint.
- Validate the SLI against reality — compare it with what users actually experienced during a known past incident. An SLI that stayed green during a real outage is measuring the wrong thing.
- Set up sampling: keep a small share of normal traffic and all of the errors and slow requests.

Phase 3 — Make problems visible fast

- Build the burn-rate dashboard, showing budget remaining, burn rates on each window, and the SLIs by service.
- Create the multi-window burn-rate alerts, mapping each to a clear action: page or ticket.
- Delete or downgrade the old cause-based alerts that nobody trusted. Reducing noise is as valuable as adding signal; a pager that lies is answered slowly.
- Write a runbook for each alert: what it means, what to check first, which switches exist, and when to escalate.

Phase 4 — Contain failures (in this exact order)

- Timeouts and deadlines everywhere first. This gives the biggest safety improvement per unit of effort, and nothing else works properly without it.
- Bulkheads next: separate pools per dependency, so one sick dependency cannot consume everything.
- Circuit breakers next, tuned per dependency from real latency and error data.
- Then retries — deliberately late in the order. Retries are only safe once timeouts, bulkheads and breakers exist to bound them. Enable at one layer, with jitter, caps and a global retry budget.
- Then backpressure: concurrency limits, bounded queues, admission control at the edge, and prioritised shedding.
- Then graceful degradation: rank the features, build the fallbacks, put every optional feature behind a switch.

Phase 5 — Prove it

- Load test to find the knee and set every limit from the measurement rather than from intuition.
- Inject faults in staging against the written hypotheses; fix what fails; re-run.
- Repeat in production with a small blast radius and a kill switch.
- Record, for every experiment, the detection time and the number of services affected. This is the 'after' measurement, taken under conditions identical to the 'before'.

Phase 6 — Keep it true

- Review the error budget on a fixed cadence and apply the policy honestly, including the unpopular parts.
- Re-run load and fault tests on every release so that regressions surface as ordinary test failures.
- After every real incident, add the scenario to the fault-injection suite so the same surprise cannot happen twice.
- Revisit the SLO periodically. If it is never at risk it may be too loose; if it is always at risk it may be unrealistic, or the system genuinely needs investment.

5.2 Workflow B — the life of a single request, in detail

This is the normal path, followed millions of times a day. Figure 3 in the next section shows the same sequence as a flowchart.

Step	What happens	Which control is acting
1	A user taps ‘buy’. The request reaches the global load balancer, which sends it to a healthy instance in the nearest region.	Health-checked routing
2	The gateway checks the caller against a rate limit and checks whether the system currently has spare capacity.	Rate limiting, admission control
3	If there is no capacity, the request is refused right here with a 429 and a Retry-After hint. It costs almost nothing and it protects the requests already inside.	Load shedding / backpressure
4	Otherwise, the gateway stamps a unique trace ID and a deadline, for example ‘answer within 800 ms’, and forwards the request.	Tracing, deadline propagation
5	The checkout service opens its own span and reads the remaining time from the deadline header.	Tracing, deadlines
6	Before calling payments, it checks the circuit breaker for that dependency. If the breaker is open, no call is made at all and the code jumps straight to the fallback.	Circuit breaker
7	If the breaker is closed, the call is made from the payment-specific thread pool, with a timeout that is the smaller of the configured value and the time actually remaining.	Bulkhead, timeout
8	If the pool is already full, the call is rejected immediately rather than queued behind work that will expire anyway.	Bulkhead backpressure
9	If the answer arrives in time, the breaker records a success and the response is returned. This is the overwhelmingly common case.	Normal path
10	If the call fails or times out, the failure is classified. A 400 is not retryable; a timeout or a 503 is.	Retry policy
11	If it is retryable, the retry budget still has room, attempts remain, and the deadline allows it, the caller waits a random time between zero and the current backoff ceiling, then tries once more.	Retry with backoff and jitter
12	If retries are exhausted or not allowed, the fallback runs: confirm the order asynchronously via the queue, or serve the cached, reduced answer.	Graceful degradation
13	The failure is counted toward the breaker’s rolling window. Enough failures and the breaker opens, so subsequent requests skip step 7 entirely.	Circuit breaker
14	The user receives an answer — full or reduced — within the deadline.	The promise is kept
15	Every span, counter and histogram from this request flows to the observability plane, where it updates the SLIs and the burn-rate dashboard within seconds.	Tracing, metrics, burn rate

5.3 Workflow C — an incident, minute by minute

The same fault, handled by the old system and by the new one. This side-by-side is the clearest illustration of what both headline numbers actually mean.

The fault: the third-party payment provider slows from 90 ms to 4 seconds

Time	Before the project	After the project
T+0:00	The provider slows down. Nothing in the system notices.	The provider slows down. Latency metrics begin moving immediately.
T+0:30	Checkout threads begin piling up waiting on payments.	Calls start hitting their timeout and failing fast, so no threads pile up. The payment pool is isolated, so checkout's other work continues.
T+1:00	The checkout thread pool is exhausted. Checkout now fails for every user, including those not paying.	The circuit breaker opens. Payment calls stop entirely, and checkout switches to asynchronous confirmation. Users still complete orders.
T+2:00	Retries at three layers multiply the load into the provider by roughly 27×, making its recovery far harder.	The fast-burn alert fires: the burn rate has crossed 14.4 over the short window. The on-call is paged. Detection is complete.
T+3:00	The basket service, which calls checkout, starts timing out. The failure has spread one hop outward.	The on-call acknowledges, opens the burn-rate dashboard, and sees that the burn is concentrated on the payment path.
T+5:00	The front end begins showing errors. Customers start contacting support.	One click from the latency spike opens an example trace. The waterfall shows a four-second span on the outbound payment call. The cause is confirmed.
T+8:00	A customer complaint reaches the team. Someone starts investigating. This is when detection actually happens.	The runbook step is executed: optional features shed, asynchronous payment confirmation confirmed as active, provider notified. Mitigation is complete.
T+12:00	Engineers are still working out which of thirty services is the origin, because everything is red at once.	Burn rate returns to normal. The alert auto-resolves. The remaining work is following up with the provider.
T+25:00	The cause is found. Mitigation begins.	—
T+40:00	Service is restored. Five services breached their SLO; a large slice of the month's error budget is gone.	—
Result	Detected at 8 minutes. Five services breached. Users saw errors.	Detected at 2 minutes. One service degraded, none breached. Users completed their purchases.

Notice that the improvement is not one clever trick. Detection improved because the alert watched the right thing on a short window. Containment improved because the timeout stopped the pile-up, the bulkhead confined it, the breaker cut the traffic, the retry budget prevented the load multiplying, and the fallback kept users served. Each control did a small job, and they compounded.

5.4 Workflow D — the ongoing weekly loop

Cadence	Activity	Purpose
Continuously	SLIs, burn rate and alerts run automatically.	Detection without human attention.
Every release	Load test and the standard fault-injection suite run in the pipeline.	Catch capacity and resilience regressions as ordinary test failures.
Weekly	Review the error budget: how much is left, what consumed it, and what the trend implies.	Apply the policy before it becomes a crisis.
Weekly	Review alerts: which fired, which were useful, which were noise. Delete or tune the noise.	Protect the trustworthiness of the pager, which is what keeps MTTD low.
Monthly	Game day: run a fault injection in production with the on-call rotation responding for real.	Practise the response and measure MTTD under known conditions.
After every incident	Blameless review; add the scenario to the fault-injection suite; fix the gaps found.	Ensure the same surprise cannot happen twice.
Quarterly	Revisit SLO targets, timeouts, pool sizes and thresholds against current traffic.	Settings age; traffic grows; last quarter's numbers slowly become wrong.

6. Architecture diagrams and a walk through every element

Four views of the same system. Figure 1 is the structure. Figure 3 is what one request does inside that structure. Figure 2 is how the structure was proven. Figure 4 is what happens when something goes wrong. Each figure is followed by a table describing every element in it.

6.1 Figure 1 — The architecture

Figure 1 — Reliability architecture: the request path, the resilience controls, and the observability plane

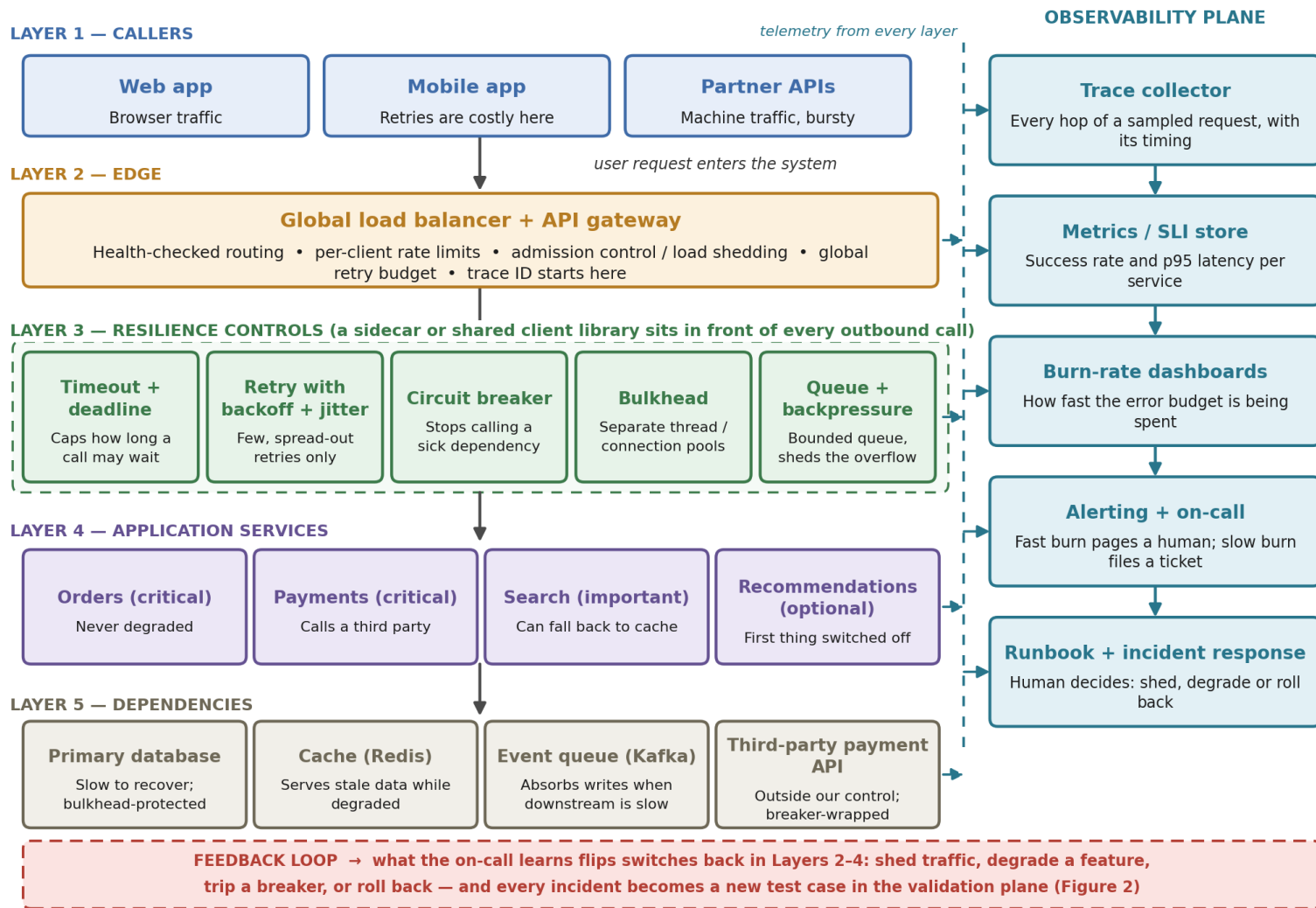


Figure 1. Requests flow top to bottom through five layers. Telemetry flows sideways into the observability plane. What is learned there flows back into the layers.

Every element of Figure 1, explained

Element	What it is	What it does for reliability
Layer 1 — Web app, Mobile app, Partner APIs	The three kinds of caller that send traffic in.	They are shown separately because they behave differently. Mobile clients are on flaky networks and retry aggressively; partner APIs arrive in machine-generated bursts. Both need firmer limits than a browser does.
Layer 2 — Global load balancer + API gateway	The single front door. Routes to healthy instances, applies per-caller rate limits, decides whether the system has capacity, and stamps the trace ID and deadline.	It is the cheapest place to say no. A request refused here has consumed almost nothing; the same request refused five hops deep has already wasted the resources that refusal was meant to protect.
Layer 3 — the dashed band	A sidecar or shared client library that wraps every outbound call. It is not a separate service; it is code sitting in the path of every call.	Placing the controls here means every service gets them consistently, and the behaviour can be changed centrally instead of being reimplemented (differently, and wrongly) in each team's code.
Timeout + deadline	A hard limit on waiting, plus the shared clock inherited from the gateway.	Stops the pile-up that turns one slow dependency into many broken services. Everything else in this layer depends on it.
Retry with backoff + jitter	Bounded re-attempts with growing, randomised waits and a global budget.	Recovers from momentary blips without multiplying load. The randomness prevents synchronised waves of retries.
Circuit breaker	Per-dependency health switch with closed, open and half-open states.	Cuts traffic to a sick dependency so callers stop wasting resources and the dependency gets room to recover.
Bulkhead	Separate thread and connection pools per dependency.	Confines the damage. A slow dependency can only exhaust its own pool, never the whole service.
Queue + backpressure	Bounded queues and concurrency limits that reject rather than accumulate.	Keeps waiting time inside the deadline and refuses work the system cannot finish in time.
Layer 4 — the four services	Application services, labelled by how important they are: critical, important, optional.	The labels are the degradation plan made concrete. Under stress, recommendations are switched off first and orders are protected last.
Layer 5 — the four dependencies	Database, cache, event queue, and a third-party API.	Each has different failure behaviour. The database is slow to recover and so is bulkheaded tightly; the cache is what makes stale-serving possible; the queue is what makes deferred writes possible; the third-party API is outside our control and always breaker-wrapped.

Element	What it is	What it does for reliability
The solid downward arrows	The request path.	Every request travels down this spine, and every hop is wrapped by the Layer 3 controls.
The dashed teal lines and the vertical bus	Telemetry leaving every layer: metrics, traces and logs.	This is deliberately drawn as a separate plane. Observability is not part of the request path; if the observability system fails, the shop must keep serving customers.
Observability — Trace collector	Reassembles the spans of one request into a single timeline.	Turns ‘something is slow’ into ‘this specific hop is slow’ in seconds.
Observability — Metrics / SLI store	Holds the success-rate and latency measurements that back the SLO.	The single source of truth for whether the promise is being kept.
Observability — Burn-rate dashboards	Shows how fast the error budget is being consumed on several time windows.	The primary detection surface. Everything else on the dashboard exists to explain what this panel is showing.
Observability — Alerting + on-call	The multi-window burn-rate rules, mapped to page or ticket.	Removes the human polling loop from detection, which is where most of the 40% improvement came from.
Observability — Runbook + incident response	The human, with a written procedure and a set of switches.	Automation contains the damage; a person decides which feature to sacrifice and when to roll back.
The red feedback band	The loop back from what was learned to what is configured.	Mitigations flip switches in Layers 2–4, and every incident becomes a new test case in the validation plane. Without this loop the architecture is a snapshot rather than a system that improves.

6.2 Figure 3 — The life of one request as a flowchart

Figure 3 — Life of a single request, and every decision the resilience controls make along the way

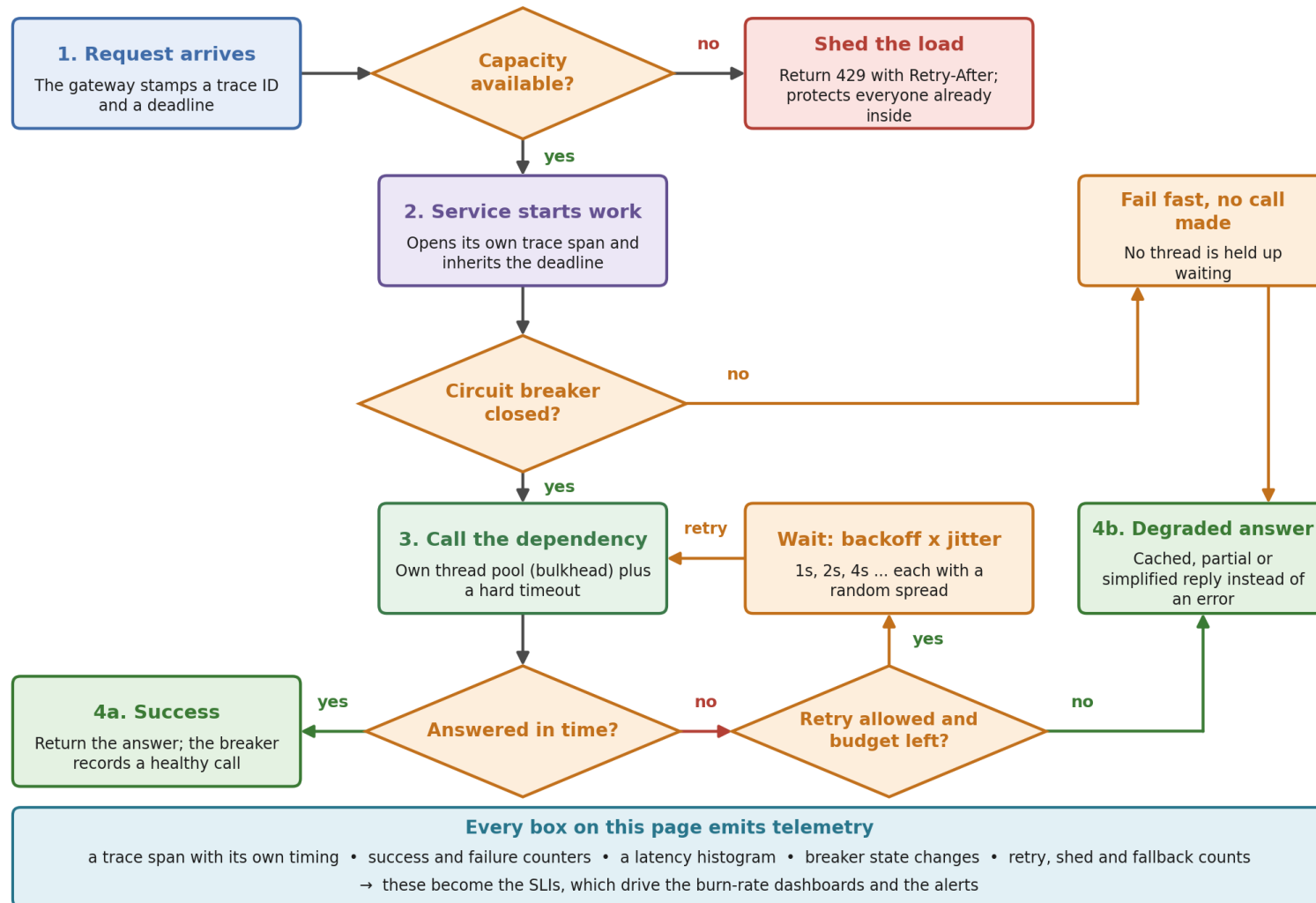


Figure 3. Diamonds are decisions. Green paths end with the user served, orange paths are protective detours, red is a deliberate refusal.

Every element of Figure 3, explained

Element	What it means	Why it is placed there
1. Request arrives	The gateway stamps a trace ID and a deadline.	Both must be created at the very front. A trace that starts halfway through cannot show where the time went, and a deadline invented later is not shared.
Capacity available?	Admission control asks whether the system can take more work right now.	First decision of all. Everything after this point costs resources, so the cheapest possible refusal has to come first.
Shed the load (429)	Refuse immediately with a Retry-After hint.	An honest, instant refusal. It protects the requests already in flight and tells the caller exactly when to come back, which prevents an immediate re-flood.
2. Service starts work	The service opens its own span and inherits the remaining time.	Inheriting rather than resetting the clock is what stops each layer from quietly granting itself a fresh, generous timeout.
Circuit breaker closed?	Is this dependency currently considered healthy?	Checked before the call, not after it fails. The entire value of a breaker is avoiding the doomed call in the first place.
Fail fast, no call made	The breaker is open, so the call is skipped entirely.	No thread is held, no connection is used, and no traffic reaches the struggling dependency. This is what containment looks like in practice.
3. Call the dependency	The real call, from a dedicated pool, with a hard timeout.	The pool bounds how much of this service can ever be tied up here; the timeout bounds how long each attempt can last.
Answered in time?	Did a usable response arrive before the deadline?	‘In time’ matters as much as ‘correct’. A correct answer that arrives after the deadline is a failure, because nobody is waiting for it any more.
4a. Success	Return the answer; the breaker records a healthy call.	The common path. Successes are counted too, because that is how a breaker knows it is safe to close again.
Retry allowed and budget left?	Four questions at once: is the operation safe to repeat, is the error the retryable kind, are attempts remaining, and is the global retry budget intact?	All four must be true. This single gate is what prevents the retry storm that used to multiply load during incidents.
Wait: backoff × jitter	Sleep for a random duration up to the current backoff ceiling, then retry once.	The growing wait gives the dependency room; the randomness stops every caller returning at the same instant.

Element	What it means	Why it is placed there
4b. Degraded answer	Cached, partial or simplified response instead of an error.	Reached from two directions: the breaker being open, and retries being exhausted or disallowed. Both roads lead to a served user rather than an error page.
The telemetry band	Every box above emits spans, counters and histograms.	Drawn as a band because it applies to every step. This data is what the SLIs, the burn-rate dashboard and the alerts are computed from.

6.3 Figure 2 — The validation plane

Figure 2 — Validation plane: how each protection was proven before real users had to depend on it

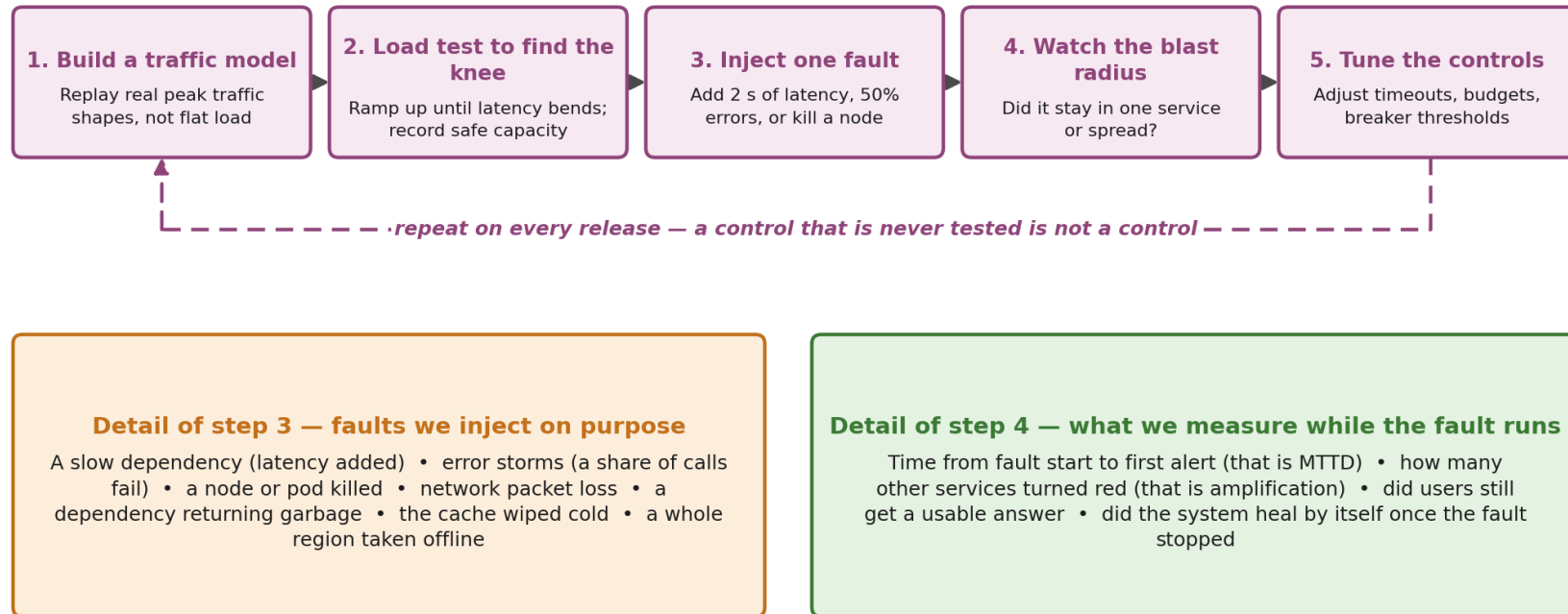


Figure 2. The loop that turns every claim about failure behaviour into a tested, repeatable fact.

Every element of Figure 2, explained

Element	What happens	Why it matters
1. Build a traffic model	Copy the real production traffic mix and shape, including bursts.	Flat synthetic load is easy to survive and proves nothing. Real traffic is spiky, and spikes are what break systems.
2. Load test to find the knee	Ramp until latency climbs steeply while throughput stops rising.	The knee is the true capacity. Every concurrency limit, pool size and queue depth is set just below it, from measurement rather than intuition.
3. Inject one fault	Break exactly one thing, deliberately, with a kill switch armed.	One fault at a time keeps the result attributable. Two at once tells you only that something went wrong.
4. Watch the blast radius	Observe whether the damage stayed in one service or spread.	This is the direct measurement of cascading-failure amplification, taken under conditions where the root cause is known for certain.
5. Tune the controls	Adjust timeouts, breaker thresholds, pool sizes and budgets in response to what was seen.	Closes the loop. The settings are outputs of the experiments, not inputs guessed beforehand.
The dashed return arrow	The whole cycle repeats on every release.	Resilience decays quietly. New code paths, new dependencies and traffic growth all erode settings that were correct last quarter.
Detail of step 3 (faults injected)	Latency, error storms, killed nodes, packet loss, corrupt responses, cold caches, region loss.	These are the failure modes that actually occur in production. Testing only for 'the process died' misses the slow-dependency case, which is the most common cause of cascades.
Detail of step 4 (what is measured)	Time to alert, number of services affected, whether users still got an answer, and whether the system self-healed.	The first two are the headline metrics. The third checks that degradation worked. The fourth checks that recovery does not require a human.

6.4 Figure 4 — Detection workflow and the containment effect

Figure 4 — The detect-and-respond workflow, and its containment effect on a cascading failure

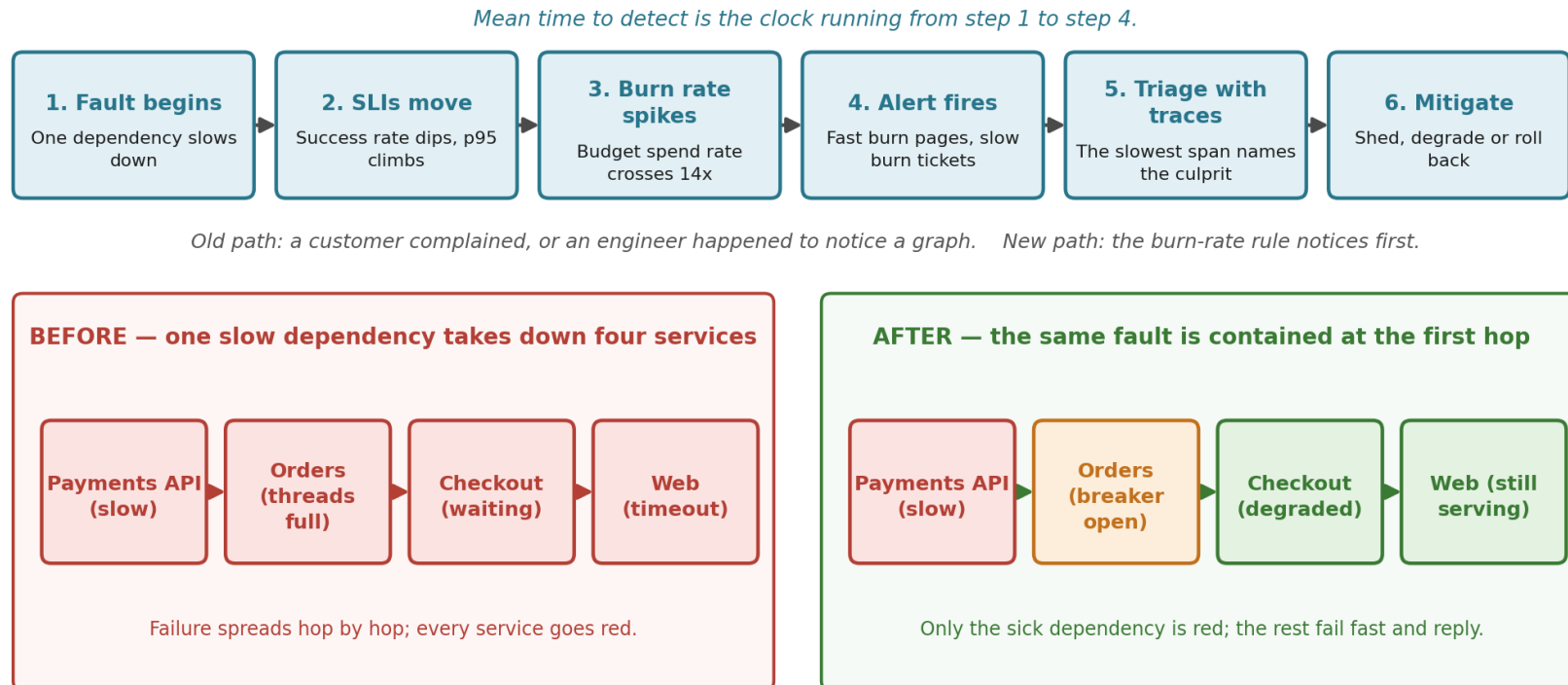


Figure 4. The top strip is the detection pipeline; the two panels below compare the same fault before and after the project.

Every element of Figure 4, explained

Element	What it means	Detail
1. Fault begins	The moment the system genuinely starts misbehaving.	The MTTD stopwatch starts here, not when someone opens a ticket.
2. SLIs move	Success rate dips or p95 latency climbs.	Because the SLIs measure the user experience, they move for any cause — a bad deploy, a slow dependency, a cold cache — without needing a rule per cause.
3. Burn rate spikes	The rate of error-budget consumption crosses a threshold.	Converting ‘errors are up’ into ‘the budget will be gone in 50 hours’ is what makes the severity obvious and non-negotiable.
4. Alert fires	Fast burn pages a human; slow burn opens a ticket.	The stopwatch stops here. Steps 1 to 4 are exactly what ‘mean time to detect’ measures.
5. Triage with traces	Open an example trace from the spike and read the waterfall.	The longest span names the culprit. This is where tracing converts a fast alert into a fast, correct diagnosis.
6. Mitigate	Shed traffic, degrade a feature, or roll back.	Mitigation comes before root-cause analysis. Stop the bleeding first; understand it afterwards.
BEFORE panel	One slow dependency, four red services.	Orders fills its threads waiting; checkout waits on orders; the web tier times out. Each hop inherits the failure, and retries at every layer multiply the load hitting the original sick dependency.
AFTER panel	One red dependency, one open breaker, two degraded-but-serving services.	The timeout stops the wait, the bulkhead confines it, the breaker cuts the traffic, and the fallback keeps users served. The fault never leaves the first hop.

7. How the improvements were actually measured

A percentage improvement is only meaningful if the before and after were measured the same way. Two measurement paths were used, and they check each other.

7.1 Measuring mean time to detect

- **Start of the clock:** the first timestamp at which the SLI shows genuine degradation, taken from the metrics store afterwards. Not the time the ticket was opened, and not the time the fix started.
- **Stop of the clock:** the moment an alert fired, or, for incidents found by a human, the first message in the incident channel.
- **Sample:** all severity-1 and severity-2 incidents in the comparison window, plus every fault-injection run, which supplies extra data points under controlled conditions.
- **Reported figure:** the mean, with the median shown alongside, because a single terrible outlier can drag a mean around. Both moved in the same direction here, which is what makes the claim credible.
- **The trap avoided:** counting only incidents that were detected. If slow-to-detect incidents are quietly reclassified as 'minor', MTTD improves on paper while getting worse in reality.

7.2 Measuring cascading-failure amplification

- **Definition used:** services that breached their own SLO during the incident, divided by services that were the root cause — with traffic amplification into the failing dependency tracked as a secondary measure.
- **Data source:** per-service SLI history plus the dependency graph reconstructed from traces, so 'affected' is a measurement rather than a memory.
- **Why fault injection is the cleaner source:** in a real incident the root cause is contested for the first twenty minutes. In an injection run it is known exactly, because it was chosen. Comparing identical injected faults before and after is a genuine like-for-like comparison.
- **The trap avoided:** comparing different fault types. A slow dependency and a crashed node spread differently, so the before and after runs used the same catalogue of faults.

7.3 Proving the SLOs were held

The claim is not just that the numbers improved, but that nothing was traded away to get there. This was checked by tracking availability and p95 latency across the whole period, including during the injection experiments, and confirming that the rolling 30-day figures never fell below target. The error budget consumed by the experiments themselves was accounted for explicitly and kept within the allowance set by the policy — which is precisely what the error budget is for.

8. Reference: the settings used, and why

These are representative values for a system of this shape. The important point is not the exact numbers but that each one is derived from a measurement and has a stated reason.

Setting	Value	Reasoning
Availability SLO	99.95% over rolling 30 days	About 21.6 minutes of budget per 30 days. High enough to matter, achievable without freezing all change.
Latency SLO	p95 under 250 ms at the edge	Measured where the user waits, so it includes queueing and internal hops.
Request deadline	800 ms end to end	Comfortably above p95, below the point where users abandon.
Per-call timeout	Just above the dependency's normal p99	A timeout should mean 'abnormal', not 'slightly unlucky'. Set per dependency, never one global value.
Retry attempts	2 extra attempts maximum	Recovers blips; well short of the point where retries become the load problem.
Backoff base and cap	100 ms base, 2,000 ms cap, doubling	Gives a struggling dependency progressively more room without exceeding the deadline.
Jitter	Full jitter: random between 0 and the current ceiling	Spreads returning traffic evenly instead of leaving spikes at the exact doubling points.
Retry budget	Retries may not exceed 10% of request volume	The global brake. Even if every local rule is followed, this prevents a retry storm.
Retry layer	Enabled at exactly one layer	Three layers retrying three times each is 27× amplification built from good intentions.
Breaker trip threshold	50% failures over a rolling 20-call minimum window	The minimum sample prevents tripping on two unlucky calls during quiet traffic.
Breaker open duration	30 seconds, then half-open	Long enough to give real relief, short enough that recovery is noticed quickly.
Breaker half-open trials	5 calls	Enough to be confident, few enough to be harmless if the dependency is still sick.
Slow-call threshold	Calls above the timeout threshold count as failures	A dependency answering 10× too slowly is as dangerous as one that is down.
Bulkhead pool size	Sized from the load-test knee, per dependency	Critical paths get larger pools; optional features get small ones so they can never starve checkout.
Queue depth	Bounded so the wait at full depth is inside the deadline	An unbounded queue converts a fast failure into a slow, useless one.
Shed response	HTTP 429 with Retry-After	Honest and actionable. It tells the caller when to return, preventing an immediate re-flood.

Setting	Value	Reasoning
Trace sampling	Small percentage of successes; 100% of errors and slow requests	Keeps cost sane while guaranteeing the interesting traces are always available.
Fast-burn alert	Burn rate above 14.4 over 1 hour, confirmed by 5 minutes	2% of the monthly budget in an hour. Worth waking someone.
Slow-burn alert	Burn rate above 6 over 6 hours; above 3 over 24 hours	Serious but not urgent. Creates a ticket instead of a page.
Budget freeze rule	Feature freeze when the budget is exhausted	Agreed in advance, in writing, so it is not negotiated during an outage.

9. Mistakes this design deliberately avoids

Each of these is a common, well-intentioned pattern that makes reliability worse. Naming them is useful, because the design choices above only make sense as answers to them.

The mistake	Why it seems sensible	What actually happens
Retrying at every layer	Each layer is trying to be helpful and resilient.	Retries multiply: three layers × three attempts is up to 27 calls at the bottom, arriving exactly when the dependency is weakest.
Retrying without jitter	Exponential backoff already spreads the load.	Every client that failed at the same instant retries at the same instant. The load returns as sharp synchronised waves.
No timeout, or a very long one	Being patient seems kinder than giving up.	Waiting consumes threads and connections. Unbounded patience is the single most common root cause of a cascade.
One shared thread pool	Simpler to configure and reason about.	Every dependency shares one fate. A slow, unimportant dependency can take down the most important path in the system.
Unbounded queues	Nothing gets dropped, so nothing is lost.	Requests wait past their deadline, so the answers are produced too late to matter. The system works hard and achieves nothing.
Alerting on CPU, memory and disk	These are easy to measure and always available.	High CPU is often fine; severe outages often happen at low CPU. The alerts are wrong in both directions, so people stop trusting the pager, and MTTD rises.
Averages instead of percentiles	The average is simple and familiar.	It hides the tail. Ninety fast requests can conceal ten disastrous ones behind a comfortable-looking mean.
Testing only the happy path	The failure paths hardly ever run.	That is precisely why they are broken. Untested fallbacks typically fail on the day they are needed.
Chasing 100% reliability	More reliable is always better.	Cost rises steeply per extra nine, users cannot perceive the difference, and all change stops — which itself causes outages.
Load testing with flat traffic	It is easy to generate and easy to reason about.	Real traffic is bursty. A system that survives a smooth ramp can still fall over at the first genuine spike.
Writing the degradation plan during the incident	Nobody wants to plan for failure in advance.	Decisions about which features to sacrifice get made under pressure, by whoever is awake, without the product owner.

10. How to explain this project out loud

A compact version for a conversation, followed by the questions that usually come next.

The 30-second version

Say this

“We had two problems: we found out about incidents too late, and one broken service used to drag several others down with it.

So we defined what ‘working’ means as a measurable promise — 99.95% of requests succeeding and 95% of them answering inside 250 milliseconds — and turned the allowed failure into an error budget with an agreed policy attached.

We instrumented every request with distributed tracing, then alerted on how fast that budget was being spent instead of on machine statistics. That cut detection time by about 40%.

Then we put timeouts, isolated pools, circuit breakers, jittered retries with a global retry budget, backpressure and graceful degradation in front of every call between services, which cut how far a failure spreads by about 35%.

And we proved all of it with load tests and deliberate fault injection rather than trusting that it would work — while holding the original latency and availability targets throughout.”

Questions that usually follow

“Which single change gave you the most?”

Timeouts and deadlines for containment, because nothing else works without them, and burn-rate alerting for detection, because it removed the human polling loop. The rest compounded on top of those two.

“How do you avoid retries making things worse?”

Retry at exactly one layer, only for idempotent operations and retryable errors, capped at two extra attempts, with full jitter, never past the deadline, never into an open breaker, and under a global budget capping retries at about 10% of traffic.

“How did you pick the circuit-breaker thresholds?”

From measurement, not intuition. Load tests gave the normal error and latency distribution per dependency; fault injection then confirmed that the breaker trips within the intended window under a real fault and closes cleanly afterwards. Both were re-tuned when the injection runs showed the breaker tripping too eagerly on quiet traffic, which is why the minimum sample size exists.

“Isn’t deliberately failing requests bad for availability?”

Shedding a small percentage for a short period costs a slice of the error budget. A full collapse costs the entire budget plus a long recovery, and it fails everyone rather than a few. The SLO is what makes that trade explicit rather than a matter of taste.

“What if the observability system itself goes down?”

It is a separate plane, deliberately not on the request path, so its failure cannot take the shop down. It is also monitored independently, because an observability system that fails silently produces the most dangerous state of all: a dashboard that is green because nothing is being measured.

“How do you keep it from decaying?”

Load and fault tests run on every release, so resilience regressions fail the pipeline like any other bug. The error budget is reviewed weekly, alert noise is pruned weekly, and every real incident is added to the injection suite so the same surprise cannot happen twice.

End of document.